

Fine-tuning LLM for code and risk evaluation

Some quick definitions

- ▶ **Reliability** is defined as the probability that a product performs its intended function without failure under specified conditions for a specified period of time. It is a function of time
- ▶ **Resilience** is defined by the capability of a system to resist, absorb and recover
 - **resist** = ability to endure disruptive events while continuing activity normally
 - **absorb** = ability to tank different disruptive events, potentially degrading performance temporarily, while still remaining resilient
 - **recover** = ability to rapidly resume activity after the end of the disruptive events
- ▶ The whole **system risk** (for a system) is defined by the sum of the probability of an event to happen multiplied by the cost caused by the event. As such, risk related to an event E is the value $R(E) = P(E) \times C(E)$, with C a positive function.

$$R = \sum_{i=1}^N (P(H_i) \times C(H_i))$$



Quick comparison of different LLM

Name	Nb.Parameters	Context Window	Open Source	MMLU Score	HellaSwag	Winogrande	GSM8K
GPT 4	1+ T billions	128k	NO	86.4 %	95.3 %	87.5 %	87.1 %
Mistral Large	-	32k	NO	81.2 %	81.3 %	71.2 %	60.7 %
Gemini 1.5 Pro	540 billions	1 million (until 10 for research)	NO	71.8 %	-	-	86.5 %
Llama	65 billions	2048	NO	69.9 %	85.8 %	80.5 %	26.7 %
Croissant	341 millions	2048	YES	25.7 %	54.2 %	55.41 %	0.61 %



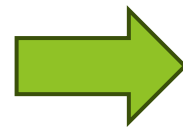
General metrics for LLMs evaluation

About MMLU score (Mean Message Length Units)

- ▶ Used to assess the average length of generated text by LLMs
- ▶ Measures how verbose or concise the language model is in its responses
- ▶ Calculated by averaging the length of messages generated by the LLM over a set of input prompts or conversations

▶ Higher MMLU scores = longer and more detailed responses

▶ lower scores suggest shorter and more succinct outputs



Quantitative measure = can be interpreted in function of the context for quality and quantity

HellaSwag

- ▶ MCQ to assess Common Sense
- ▶ Metric = percentage of good answers over the dataset
- ▶ Trivial for human but difficult for machines
- ▶ Size :
 - ▶ Train : 39905
 - ▶ Validation : 10042
 - ▶ Test : 10003

How to catch dragonflies. Use a long-handled aerial net with a wide opening. Select an aerial net that is 18 inches (46 cm) in diameter or larger. Look for one with a nice long handle.

a) Loop 1 piece of ribbon over the handle. Place the hose or hose on your net and tie the string securely.

b) Reach up into the net with your feet. Move your body and head forward when you lift up your feet.

c) If possible, choose a dark-colored net over a light one. Darker nets are more difficult for dragonflies to see, making the net more difficult to avoid.

d) If it's not strong enough for you to handle, use a hand held net with one end shorter than the other. The net should have holes in the bottom of the net.

WinoGrande

- ▶ Two choice fill-in-the blank that created such that statistics only is not good enough
- ▶ Metric = percentage of good answers over the dataset
- ▶ Consist of multiple pair of questions where one word were changed such that the answer was reversed
- ▶ Size : 44 000

Twin sentences		Options (answer)
✗	The monkey loved to play with the balls but ignored the blocks because he found them exciting.	balls / blocks
	The monkey loved to play with the balls but ignored the blocks because he found them dull.	balls / blocks
✗	William could only climb beginner walls while Jason climbed advanced ones because he was very weak.	William / Jason
	William could only climb beginner walls while Jason climbed advanced ones because he was very strong.	William / Jason
✓	Robert woke up at 9:00am while Samuel woke up at 6:00am, so he had less time to get ready for school.	Robert / Samuel
	Robert woke up at 9:00am while Samuel woke up at 6:00am, so he had more time to get ready for school.	Robert / Samuel
✓	The child was screaming after the baby bottle and toy fell. Since the child was hungry, it stopped his crying.	baby bottle / toy
	The child was screaming after the baby bottle and toy fell. Since the child was full, it stopped his crying.	baby bottle / toy

gsm8k

- ▶ Middle school math problems
- ▶ Metric = percentage of good answers over the dataset
- ▶ A good answer is when the Answer after the “A:” is good
- ▶ Asses how good a llm is in basic math
- ▶ Size : 8500

Dean's mother gave him \$28 to go to the toy store. Dean bought 6 toy cars and 5 teddy bears. Each toy car cost \$2 and each teddy bear cost \$1. His mother then feels generous and decides to give him an extra \$10. How much money does Dean have left?

Ground Truth:

The cost of the toy cars is $6 \text{ cars} \times \$2/\text{car} = \12 .

The cost of the teddy bears is $5 \text{ bears} \times \$1/\text{bear} = \5 .

The total cost of the toys is $\$12 + \$5 = \$17$.

Adding the two amounts of money his mother gave him, we find that Dean has $\$28 + \$10 = \$38$ to spend.

Dean has $\$38 - \$17 = \$21$ left.

A: 21



Code-Related metrics for LLMs evaluation

A metric: pass@k

- ▶ The most widely used metric for code evaluation
- ▶ Used in many benchmarks
- ▶ The pass@k metric is defined as the probability that at least one of the top k-generated code samples for a problem passes the unit tests.

$$\text{pass}@k := \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

```
def pass_at_k(n, c, k):  
    """  
    :param n: total number of samples  
    :param c: number of correct samples  
    :param k: k in pass@$k$  
    """  
    if n - c < k: return 1.0  
    return 1.0 - np.prod(1.0 - k /  
                          np.arange(n - c + 1, n + 1))
```

Figure 3. A numerically stable script for calculating an unbiased estimate of pass@k.

HumanEval (2021)

- ▶ A general coding dataset
- ▶ Metric = pass@k
- ▶ Interview question-style
- ▶ Size : 164 coding challenges
- ▶ Comes with a pipeline to test your LLM
- ▶ One of the most widely used benchmarks for code

```
def solution(lst):  
    """Given a non-empty list of integers, return the sum of all of the odd elements  
    that are in even positions.  
  
    Examples  
    solution([5, 8, 7, 1]) ==>12  
    solution([3, 3, 3, 3, 3]) ==>9  
    solution([30, 13, 24, 321]) ==>0  
    """  
    return sum(lst[i] for i in range(0, len(lst)) if i % 2 == 0 and lst[i] % 2 == 1)
```

MBPP(2021)

- ▶ Python programming problems, designed to be solvable by entry-level programmers
- ▶ Metric = pass@k
- ▶ Programming fundamentals, standard library functionality, and so on. Each problem consists of a task description, code solution and 3 automated test cases.
- ▶ Size : 1000 coding challenges
- ▶ Each problem consists of a task description, code solution and 3 automated test cases.
- ▶ 81 citations in 2024

prompt

Write a python function to check if a given number is one less than twice its reverse. Your code should satisfy these tests:

```
assert check(70) == False
assert check(23) == False
assert check(73) == True
```

model

```
def check(n):
    if n == 2*int(str(n)[::-1])-1:
        return True
    else:
        return False
```

prompt

Write a function to find the smallest missing element in a sorted array. Your code should satisfy these tests:

```
assert smallest_missing([0, 1, 2, 3, 4, 5, 6], 0, 6) == 7
assert smallest_missing([0, 1, 2, 6, 9, 11, 15], 0, 6) == 3
assert smallest_missing([1, 2, 3, 4, 6, 9, 11, 15], 0, 7) == 0
```

model

```
def smallest_missing(arr, n, m):
    smallest = min(n, m)
    for i in range(n, m + 1):
        if arr[i] <= smallest:
            smallest += 1
    return smallest
```

DS-1000

- ▶ A data science coding dataset
- ▶ Metric = pass@k
- ▶ Data science (numpy...) challenges
- ▶ Size : 1000 challenges
 - ▶ Pandas: 291
 - ▶ NumPy: 220
 - ▶ Matplotlib: 155
 - ▶ ...

Problem:

I could not find a built-in function in Python to generate a log uniform distribution given a min and max value (the R equivalent is here), something like: `loguni[n, min, max, base]` that returns `n` log uniformly distributed in the range `min` and `max`.

The closest I found though was `numpy.random.uniform`.

That is, given range of `x`, I want to get samples of given size (`n`) that suit log-uniform distribution.

Any help would be appreciated!

A:

```
<code>
import numpy as np
min = 1
max = np.e
n = 10000
</code>
BEGIN SOLUTION
<code>
[insert]
</code>
END SOLUTION
<code>
print(result)
</code>
```

Reference Solution

```
import scipy.stats
result = scipy.stats.loguniform.rvs(a = min, b = max, size = n)
```

Automatic Evaluation

Test case 1

```
min = 1
max = np.e
n = 10000
ans = ... # generated by Reference solution
```

NumPy

Test code

```
np.testing.assert_array_equal(result.shape, ans.shape)
from scipy.stats import ks_2samp
# Kolmogorov-Smirnov Test judges whether the two sampled
# from similar distribution
assert ks_2samp(result, ans)[0] <= 0.1
```

Surface form constraints

`for` and `while` should not appear in Syntax Tree

NumPy example problem involving randomness, requiring the use of a specialist knowledge test.

A potential improvement: beyond@k (2024)

- Comes along with a benchmark: Mercury
- Assesses efficiency more than correctness
- Problems extracted from leetcode
- Size:
 - Training: 1,633
 - Evaluation: 256

$$\text{Beyond}@K = \frac{1}{K} \sum_{k=0}^{K-1} \left[1 - \frac{\max(R_A) - R_k}{\max(R_A) - \min(R_A)} \right]$$

